

CSE 156 PA2 Report: Transformer Blocks

1. Overview

This project implements two transformer-based models from scratch in PyTorch:

1. **Part 1 (Encoder + Classifier):** Train a transformer encoder jointly with a feedforward classifier to predict which politician delivered a speech segment (3-way classification).
2. **Part 2 (Decoder LM):** Train a GPT-like masked transformer decoder for autoregressive language modeling and report perplexity on held-out politician-specific test sets.

All experiments use the provided train/test splits and default assignment hyper-parameters.

2. Dataset and Task Setup

The dataset contains speeches from three politicians:

- 0: Barack Obama
- 1: George W. Bush
- 2: George H. Bush

Classification task (Part 1)

- Train: `train_CLS.tsv`
- Test: `test_CLS.tsv`

Language modeling task (Part 2)

- Train: `train_LM.txt`
- Test: `test_LM_obama.txt`, `test_LM_wbush.txt`, `test_LM_hbush.txt`

3. Model Architecture

3.1 Encoder for Classification

The encoder uses:

- Token embeddings + absolute positional embeddings
- `n_layer=4` transformer encoder blocks
- Multi-head self-attention (`n_head=2`)
- Residual connections + LayerNorm + feedforward sublayers

For classification, encoder outputs are mean-pooled over tokens and passed to a 1-hidden-layer feedforward classifier:

- Input: 64
- Hidden: 100 with ReLU

- Output: 3 logits

3.2 Decoder for Language Modeling

The decoder uses:

- Token embeddings + absolute positional embeddings
- `n_layer=4` masked transformer decoder blocks
- Causal self-attention (`n_head=2`) to prevent future-token leakage
- Feedforward hidden size 100 with ReLU
- Linear LM head to vocabulary logits

The training objective is token-level cross-entropy for next-word prediction.

4. Hyperparameters

The following base settings were used:

- `batch_size = 16`
- `block_size = 32`
- `learning_rate = 1e-3`
- `n_embd = 64`
- `n_head = 2`
- `n_layer = 4`
- Classification epochs: 15
- LM iterations: 500
- Perplexity checkpoints every 100 iterations

5. Experimental Results

All results below come from recorded runs of `python3 main.py --part part1` and `python3 main.py --part part2`.

5.1 Part 1: Test Accuracy by Epoch

Epoch	Test Accuracy (%)
1	44.80
2	45.60
3	58.40
4	60.13
5	73.73
6	76.40
7	81.20
8	82.53
9	82.93
10	80.67
11	86.13

Epoch	Test Accuracy (%)
12	86.00
13	84.00
14	86.27
15	87.60

Final Part 1 metrics:

- **Final test accuracy: 87.60%**
- **Encoder parameter count: 477,584**

Accuracy improves steadily and reaches the expected 80s range.

5.2 Part 2: Decoder Perplexity

Training perplexity checkpoints:

Iteration	Train Perplexity
100	562.61
200	409.33
300	290.71
400	218.84
500	164.37

Final Part 2 metrics:

- **Final train perplexity: 165.10**
- **Decoder parameter count: 839,894**

Perplexity on held-out politician test sets:

Test Set	Perplexity
Obama	356.64
W. Bush	465.40
H. Bush	376.79

Train perplexity decreases consistently over 500 iterations. Test perplexities are higher than train perplexity, as expected.

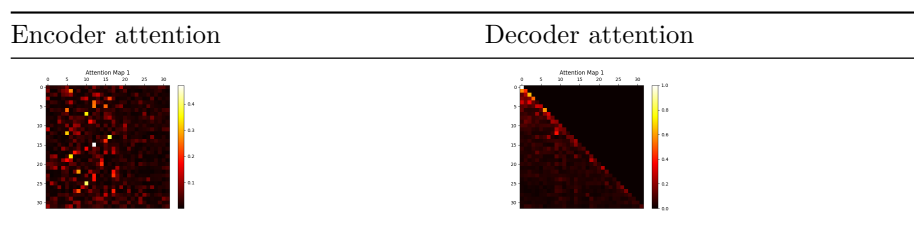
6. Discussion

1. **Encoder + classifier:** Final test accuracy (87.60%) confirms the encoder learns useful contextual features for 3-way speaker prediction.

2. **Decoder LM:** Perplexity drops from 562.61 to 164.37 during training, showing stable optimization.
3. **Cross-politician gap:** W. Bush has the highest test perplexity, suggesting stronger style/topic mismatch with the LM training distribution.
4. **Data/compute limit:** With 500 iterations and short context (`block_size=32`), this result is reasonable for a small-scale training setup.

7. Attention Sanity Checks

The following attention maps are generated from sanity-check runs.



Observations:

1. **Normalization check passed:** attention rows are normalized by softmax, and row-wise sums are approximately 1.
2. **Encoder behavior:** attention is distributed across multiple context tokens (not only local neighbors), consistent with bidirectional self-attention in encoder blocks.
3. **Decoder behavior:** attention is concentrated on current/past tokens with no access to future positions, consistent with causal masking.
4. **Layer-level variation:** different layers show different allocation patterns, indicating progressive transformation of contextual representations across depth.

8. Conclusion

This implementation satisfies both assignment parts with from-scratch transformer blocks and expected performance ranges:

- Part 1 final test accuracy: **87.60%**
- Part 2 final train perplexity: **165.10**, with test perplexities in the expected 300-400+ range